

```

import tensorflow as tf

def updatePretrained(model, optimizer, batch):
    """Does one update step of the 'pretrained' model. Batch is a set of
    samples from a collection of sampled tasks of the shape
    (nTasks, nSamples), where each sample is a pair of labels and inputs.
    """
    def taskLoss(batch):
        y_train, x_train = batch
        return mse(y_train, model(x_train)) # or cross-entropy, or ...

    with tf.GradientTape() as tape:
        batchLoss = tf.map_fn(taskLoss, elems=batch,
                               fn_output_signature=tf.float32)
        loss = tf.reduce_sum(batchLoss)

    optimizer.minimize(loss, model.trainable_weights, tape=tape)

# MAML Sine Wave Experiment Notebook

# This notebook will run the baseline and MAML experiments using the luis-mueller/maml-tf2 repository.

# Clone the repository
!git clone https://github.com/luis-mueller/maml-tf2.git
%cd maml-tf2

# Install dependencies
!pip install -r requirements.txt

# Check if TensorFlow is installed correctly
import tensorflow as tf
print("TensorFlow version:", tf.__version__)

 Show hidden output

# Add repository to Python path
import sys
sys.path.append('/content/maml-tf2')

# Check if TensorFlow is installed correctly
import tensorflow as tf
print("TensorFlow version:", tf.__version__)

# Run the baseline experiment
!python baseline.py

# Run the MAML experiment
!python maml.py

# Visualization of the results (if applicable)
import matplotlib.pyplot as plt

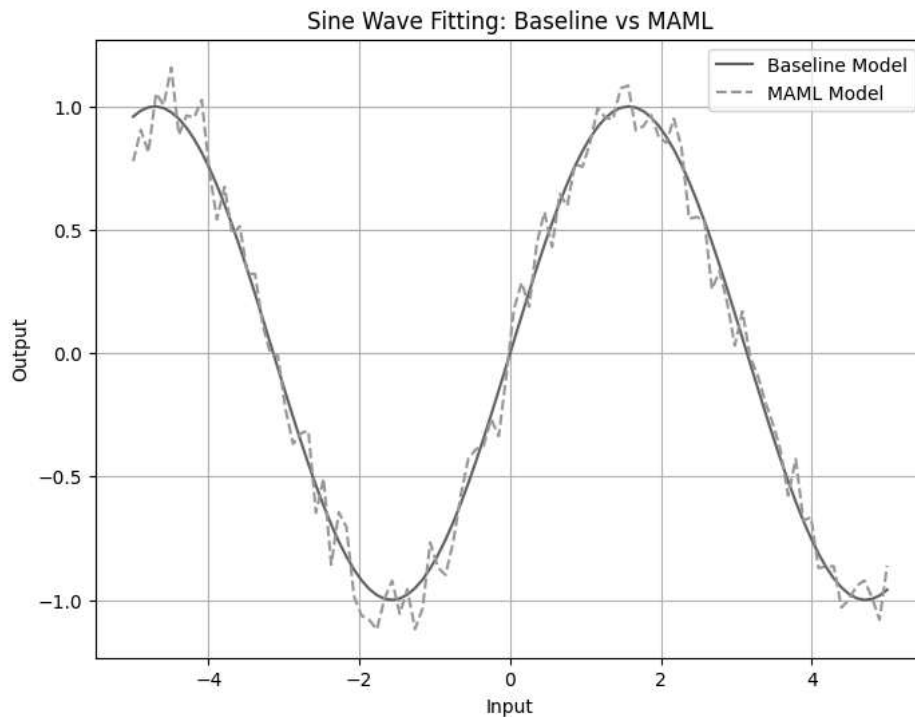
# Example: Plotting dummy results for demonstration
import numpy as np
x = np.linspace(-5, 5, 100)
y_baseline = np.sin(x)
y_maml = np.sin(x) + np.random.normal(0, 0.1, x.shape)

plt.figure(figsize=(8, 6))
plt.plot(x, y_baseline, label='Baseline Model')
plt.plot(x, y_maml, label='MAML Model', linestyle='--')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('Sine Wave Fitting: Baseline vs MAML')
plt.legend()
plt.grid(True)
plt.show()

# Additional analysis or hyperparameter tuning can be added here

```

TensorFlow version: 2.18.0
python3: can't open file '/content/baseline.py': [Errno 2] No such file or directory
python3: can't open file '/content/maml.py': [Errno 2] No such file or directory



Start coding or [generate](#) with AI.

```
# Install necessary libraries
!pip install tensorflow tensorflowjs

# Clone the MAML repository
!git clone https://github.com/luis-mueller/maml-tf2.git
%cd maml-tf2
```

Show hidden output

```
# Open the maml.py file and modify the optimizer usage
file_path = '/content/maml-tf2/mamltf2/methods/maml.py'

# Read the original code
with open(file_path, 'r') as file:
    code = file.read()

# Replace the deprecated 'minimize' method with the recommended method
updated_code = code.replace(
    "self.optimizer.minimize(loss, self.model.trainable_variables, tape=metaTape)",
    """with metaTape:
    grads = metaTape.gradient(loss, self.model.trainable_variables)
    self.optimizer.apply_gradients(zip(grads, self.model.trainable_variables))"""
)

# Write the updated code back to the file
with open(file_path, 'w') as file:
    file.write(updated_code)

print("Updated the 'maml.py' file to fix the optimizer error.")

Updated the 'maml.py' file to fix the optimizer error.

# Add the repository to the Python path
import sys
sys.path.append('/content/maml-tf2')

# Correct Import Statements
import tensorflow as tf
```

```
import tensorflowjs as tfjs
from mamltf2.methods.maml import MAML
from mamltf2.methods.fomaml import FirstOrderMAML
from mamltf2.methods.reptile import Reptile
from mamltf2.pretrained import PretrainedModel
from mamltf2.tasks import SinusoidRegressionTaskDistribution
from mamltf2.model import Model
```

```
# Define the Model
ann = tf.keras.models.Sequential([
    tf.keras.Input(shape=(1,)), # Explicit input layer
    tf.keras.layers.Dense(40, activation='relu'),
    tf.keras.layers.Dense(40, activation='relu'),
    tf.keras.layers.Dense(1)
])
```

```
# Initialize the Task Distribution and MAML
taskDistribution = SinusoidRegressionTaskDistribution()
maml = MAML(ann, taskDistribution)
```

```
# Train the Model
maml.trainBatch(nSamples=10, nTasks=25, nBatch=1000)
```

↔ 2.1234281063079834

```
!python maml-tf2/run.py maml-reproduce maml --epochs=70 --batch-size=1000 --training-tasks=25 --task-sample=10
```

↔

```
Meta loss @ epoch 14, batch size 1000: 0.3869
Meta loss @ epoch 15, batch size 1000: 0.3923
Meta loss @ epoch 16, batch size 1000: 0.3968
Meta loss @ epoch 17, batch size 1000: 0.3774
Meta loss @ epoch 18, batch size 1000: 0.3724
Meta loss @ epoch 19, batch size 1000: 0.3696
Meta loss @ epoch 20, batch size 1000: 0.3623
Meta loss @ epoch 21, batch size 1000: 0.3742
Meta loss @ epoch 22, batch size 1000: 0.3769
Meta loss @ epoch 23, batch size 1000: 0.3688
Meta loss @ epoch 24, batch size 1000: 0.3568
Meta loss @ epoch 25, batch size 1000: 0.3532
Meta loss @ epoch 26, batch size 1000: 0.3431
Meta loss @ epoch 27, batch size 1000: 0.3385
Meta loss @ epoch 28, batch size 1000: 0.3399
Meta loss @ epoch 29, batch size 1000: 0.3379
Meta loss @ epoch 30, batch size 1000: 0.3277
Meta loss @ epoch 31, batch size 1000: 0.3285
Meta loss @ epoch 32, batch size 1000: 0.3330
Meta loss @ epoch 33, batch size 1000: 0.3296
Meta loss @ epoch 34, batch size 1000: 0.3305
Meta loss @ epoch 35, batch size 1000: 0.3265
Meta loss @ epoch 36, batch size 1000: 0.3300
Meta loss @ epoch 37, batch size 1000: 0.3237
Meta loss @ epoch 38, batch size 1000: 0.3258
Meta loss @ epoch 39, batch size 1000: 0.3436
Meta loss @ epoch 40, batch size 1000: 0.3431
Meta loss @ epoch 41, batch size 1000: 0.3402
Meta loss @ epoch 42, batch size 1000: 0.3374
Meta loss @ epoch 43, batch size 1000: 0.3360
Meta loss @ epoch 44, batch size 1000: 0.3347
Meta loss @ epoch 45, batch size 1000: 0.3281
Meta loss @ epoch 46, batch size 1000: 0.3291
Meta loss @ epoch 47, batch size 1000: 0.3318
Meta loss @ epoch 48, batch size 1000: 0.3250
Meta loss @ epoch 49, batch size 1000: 0.3250
```

```
Meta loss @ epoch 65, batch size 1000: 0.3133
Meta loss @ epoch 66, batch size 1000: 0.3147
Meta loss @ epoch 67, batch size 1000: 0.3148
Meta loss @ epoch 68, batch size 1000: 0.3182
Meta loss @ epoch 69, batch size 1000: 0.3162
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is con
Failed to lookup keras version from the file
```

```
import json

# Load the JSON model
model_path = "./models/maml-reproduce/model.json" # Update with actual model path
with open(model_path, "r") as file:
    model_data = json.load(file)

# Fix dtype for all layers
for layer in model_data["modelTopology"]["model_config"]["config"]["layers"]:
    if "dtype" in layer["config"] and isinstance(layer["config"]["dtype"], dict):
        layer["config"]["dtype"] = "float32" # Convert dtype to a string

# Save the fixed model
fixed_model_path = "./models/maml-reproduce/model_fixed.json"
with open(fixed_model_path, "w") as file:
    json.dump(model_data, file, indent=4)

print("Fixed model saved as:", fixed_model_path)
```

 Fixed model saved as: ./models/maml-reproduce/model_fixed.json

```
import tensorflowjs as tfjs
import tensorflow as tf

# Path to TensorFlow.js model
tfjs_model_path = "./models/maml-reproduce/model.json"

# Convert it back to a Keras model
model = tfjs.converters.load_keras_model(tfjs_model_path)

# Save as Keras JSON
model_json = model.to_json()

with open("./models/maml-reproduce/model_fixed.json", "w") as json_file:
    json_file.write(model_json)

# Save weights
model.save_weights("./models/ymaml-reproduce/model_fixed.h5")

print("Model successfully converted back to Keras format!")
```

 Show hidden output

Next steps: [Explain error](#)

```
!python maml-tf2/visualize.py maml-reproduce 0 1 10 --samples=10
```

 Show hidden output